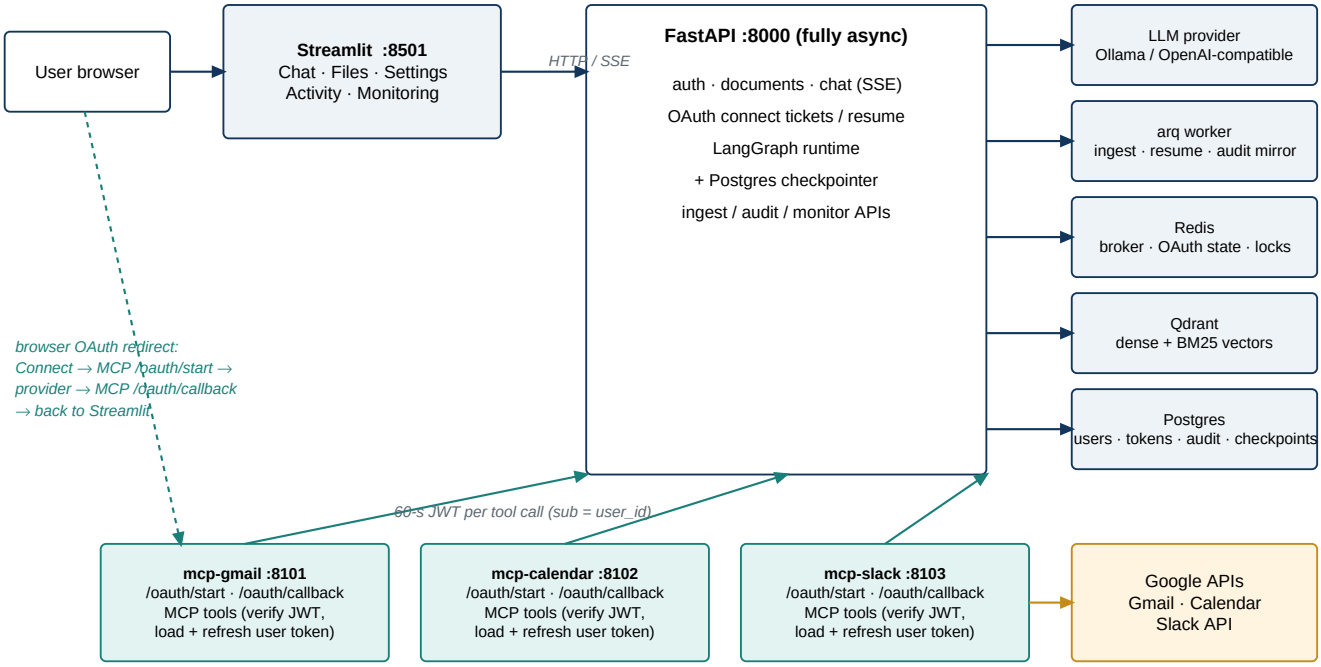
Banking Multi-Agent RAG Chatbot — Design Document

LangGraph · MCP (Gmail, Calendar, Slack) · per-user OAuth2 · Excel/CSV-only RAG · FastAPI · Streamlit | Live demo: https://handbrake-raking-embody.ngrok-free.dev (API docs at /docs; tunnelled from my machine)

1. System Architecture



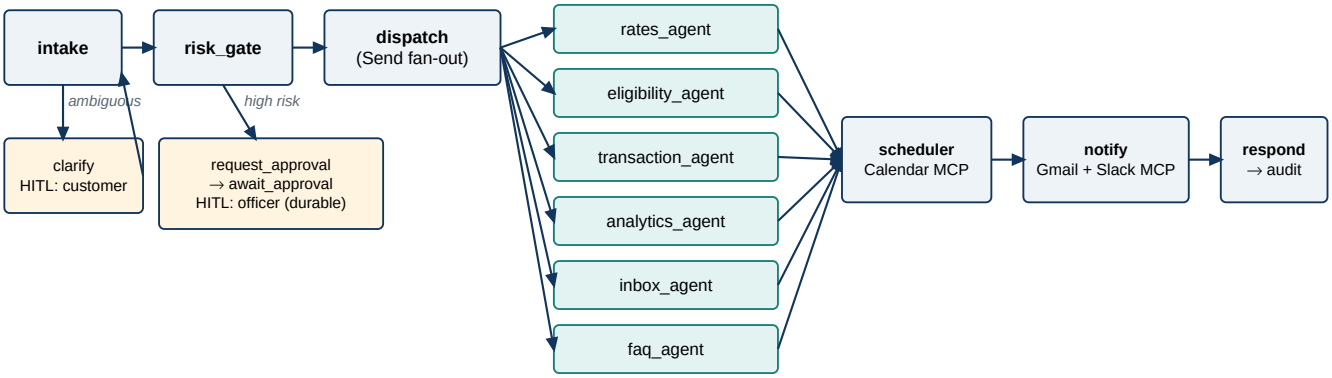
Request path. The browser talks only to Streamlit. Streamlit holds the user's session JWT in `st.session_state` and calls FastAPI over HTTP; chat uses server-sent events (`step | token | final | interrupt`) so the UI can show which agent is working. FastAPI runs the LangGraph graph in-process with a Postgres checkpointer; heavy or deferred work (embedding, resuming a graph after an officer approves, mirroring audit summaries to Slack) goes through **arq on Redis** to a worker. Agents reach Gmail/Calendar/Slack only through three MCP servers over streamable HTTP; each call carries a 60-second JWT (`sub=user_id, aud=mcp:<service>`) minted by the API from the *graph state's* user id, never from model output. The MCP server, not the API, loads and decrypts that user's token.

Process documentation. Step-by-step descriptions of the six mandatory processes (file ingestion, query routing, aggregated queries, OAuth2 token lifecycle, MCP tool invocation, audit logging) with components, data formats, error handling and a sequence diagram each are in **Appendix A**.

2. RAG Pipeline Design (Excel & CSV only)

Stage	Design and reason
Boundary	Enforced at two layers (widget + `validate_upload`) and again implicitly: `load_tables` is the *only* reader and only understands pandas CSV / openpyxl. Nothing else can reach the index.
Parsing	All cells read as strings (no silent type coercion of IDs/dates); headers normalised; empty rows/columns dropped; encoding fallback (utf-8-sig → latin-1); row cap (100k) rejects abusive files. **Every sheet** of a workbook is its own table; each is tagged with file + sheet in every chunk so a 'Home Loan' sheet and a 'Personal Loan' sheet never blur.
Table classification	Heuristic on column names → rates / fees / compliance / eligibility_rules / profile / transactions / other. Used to filter retrieval (the rates agent never reads transaction rows) and to pick the chunk strategy.
Chunking	Table-aware, three kinds: **schema** chunk per sheet (columns, numeric ranges, categorical values — answers 'what does this file contain'); **row** chunks rendered `[file > sheet] col: value; …` (one row per chunk, grouped ≤40 only for very large tables so a 100k-row file stays within MAX_ROW_CHUNKS); **monthly summary** chunks for transaction tables. Rows are kept intact because a half-row is meaningless.
Embedding	**Hybrid**: dense `BAAI/bge-small-en-v1.5` (384-d, ONNX via fastembed, local) + sparse `Qdrant/bm25`. Dense captures meaning ('mortgage' ≈ 'home loan'); BM25 keeps exact tokens ('750', 'APT-2026…', 'BR001') that dense models blur — essential for tabular data. Local models mean no customer data leaves the host to embed.
Retrieval	Qdrant hybrid query with **RRF fusion**, filtered in the store by `scope=bank OR (scope=customer AND owner_id=caller)`; callers cannot widen it. Retrieved rows are numbered `[S#]` in the prompt; the model must cite tags or say 'not found'. Sources (file, sheet, row range) are shown under each answer.
Exact reads	Numbers are never left to an LLM: profile fields, transaction anomaly flags (large transfer vs the threshold read from the compliance CSV, 3σ outliers, rapid succession, repeated amounts) and all aggregates are computed with pandas (Section 4).

3. Multi-Agent Graph Design (LangGraph)



Conditional edges: intake→clarify|risk_gate · risk_gate→respond|request_approval|specialists · approval→respond|specialists.
Specialists run in parallel; evidence merges into state; clarify and approval pause the thread via interrupt().

- **Nodes:** intake, clarify, risk_gate, request_approval, await_approval, dispatch (router), six specialists (rates, eligibility, transaction, **analytics**, inbox, faq), scheduler, notify, respond, audit. **Conditional edges** implement real branching: ambiguity → clarify loop; out-of-scope → respond; high risk → approval; `dispatch` returns a list of `Send` objects so the needed specialists run **in parallel** (a loan-rate + appointment message runs rates + eligibility concurrently, then scheduler, then notify).
- **State** (`BankState`): identity fields set by the API (user_id, email, tz), `messages` (add_messages reducer), per-turn working memory reset by intake (`evidence` merge-dict so parallel agents write disjoint keys; `actions`/`trace` append-lists), approval and appointment records. Multi-turn memory = the checkpointed thread.
- **Human-in-the-loop:** (1) *customer* — ambiguous requests (e.g. a transfer with no amount) call `interrupt()` in `clarify`; the next chat message resumes the graph. (2) *officer* — transfers ≥ the compliance threshold, fraud reports, large disputes create an `approvals` row, alert #compliance-alerts, and pause durably in `await_approval` (state lives in Postgres, survives restarts). An officer decides in the UI (four-eyes: not their own request); the API enqueues a worker job that resumes the thread and posts the outcome to the customer's conversation. The bot never claims to move money.
- **Failure modes:** every MCP call returns `McpResult(ok, error)` and never raises for expected failures; a Calendar timeout → `appointment.status=failed`, the reply says so, other actions still run; a missing connection is reported as *skipped* with a 'connect it in the sidebar' message (never silent). Scheduling is idempotent (event id derived from the booking reference) and double-booking is prevented by a DB reservation under a per-RM advisory lock plus a Calendar availability re-check inside `create_event`. A Redis lock keeps one run per thread. Dates like 'this Thursday' are resolved in code (`resolve_relative_date`), not by the model.

4. Aggregated Query Handling (process + answers to the assessment's questions)

- **Detection.** Intake classifies `analytics_query` (totals, counts, averages, rankings, trends, comparisons); a regex backstop (aggregate word + a parsed period) catches phrasing small models miss. **Structured compute, not LLM arithmetic:** an LLM *planner* fills a typed `AggPlan` (table, date column, filters, group-by incl. derived month/quarter/year, metrics sum/count/count_distinct/avg/min/max, join, denominator, comparison) for the 4 most relevant tables; `pandas` executes it over **every row** of the full files; the reply text is generated from the numbers, not by the LLM. Metric cards, a table and a chart are rendered from the same result.
- **Dates** come from a deterministic parser (`periods.py`: Q1 2026, April to June, last quarter, first half, last 3 months, financial year with configurable start month) — a wrong date silently changes a banking total, so the model never writes one; model-written date filters are discarded.
- **Multi-file questions.** *Join:* 'branch' lives in Accounts, loans only have account_id → the plan joins on the shared key (left join, right side de-duplicated, noted if keys repeat). *Ratio:* 'recovered as % of total outstanding' = sub-query A / sub-query B over two files (`denominator`, with or without the period). *Comparison:* 'top branch Q1 vs Q4' computes the top group, then the same groups in the previous/second period (delta and %). Multi-sheet workbooks with identical columns also expose a combined `file::*` table.
- **Guardrails against hallucinated aggregates.** (1) Plans are validated against the real schema — unknown table/column ⇒ error fed back for *one* retry, then an explicit 'I couldn't compute that exactly' (no estimate). (2) `repair_plan` fixes predictable model slips from the question text (prose in a numeric filter, counting instead of summing, identifier group-by, missing join) and every repair is shown to the user under 'How this was computed'. (3) Zero matches, a period beyond the data's date coverage, and un-filtered statuses are stated explicitly; empty months are shown as 0 only up to the data's last month. (4) Filter values are parsed strictly (Rs / lakh / crore) — the first live test showed a lenient parser reading prose as 0.1.
- **Accuracy/latency trade-off.** Exactness costs one planner LLM call (+ one retry on invalid plans) and a full-file pandas pass (~tens of ms for 100k rows) instead of a single retrieval; with a local 14B model on CPU planning dominates

(~1.5-3 min), with a hosted model seconds. The alternative — retrieving top-K rows and letting the LLM add them up — is faster but wrong by construction, which is unacceptable where an aggregate has compliance consequences. **Verified:** results were compared with independent pandas calculations on the supplied dataset (e.g. Apr–Jun debit 48,190,492.81 / credit 53,495,177.93 month by month; top categories; overdue count) and 30+ unit tests cover each query class.

5. MCP Integration Design

Server	Tools	Why it adds value (not just presence)
Gmail	send_email · list_recent_emails · read_email	Confirmation to the customer, high-priority notice to the RM, summarising a customer's recent mail (content treated as *untrusted data* in prompts).
Calendar	check_availability · find_free_slots · create_event	Availability check before booking; free-slot suggestions on the requested day; invite with branch address, reference number and agenda; conflict ⇒ alternatives, never a double-booking.
Slack	post_message · post_structured · notify_user_by_email	RM DM, #loans-team lead, #compliance-alerts for approvals/high-risk, structured summary of every resolved query to #audit-log. Slack control sequences (<!channel>) are escaped.

- **Tool-selection policy** is explicit, not left to a free-form model: *intents* (extracted by intake) decide which node runs — `schedule_appointment` → scheduler (Calendar); the notify node maps intents/risk to a fixed set (customer email ⇒ Gmail; RM + team ⇒ Slack; high risk ⇒ Slack compliance) and runs them **concurrently** with `asyncio.gather`. This is auditable and testable; free tool-calling would let a prompt-injected email trigger actions.
- **Token injection:** graph → `call_tool(service, state.user_id, tool, args)` → short-lived JWT → MCP server verifies, loads *that* user's token (refreshing if needed) → provider API. Tools accept no user/token arguments. **Failures** (unreachable, not connected, provider error) become `failed|skipped` actions with a human-readable reason shown in the reply and the Activity log; every call is audited (service, tool, outcome, latency). A freshly connected Slack workspace has no `#loans-team` etc.: a post to a missing channel falls back to the workspace's general channel and names the intended one, so an alert is never lost.

6. OAuth2 Architecture & Token Lifecycle

Two different things. The *OAuth client* (ID + secret) identifies this **application** to Google/Slack and is required to start any consent flow; it grants access to nobody's data. A *user token* is what a person grants when they sign in. The system has exactly one client per provider and **one token per user per service** — there is no service account, no shared mailbox and no token used on behalf of more than one user.

Step	What happens (all server-side)
0 App client	A bank officer opens **Integrations setup**, which lists the exact redirect URIs and scopes to register, and pastes the client ID + secret. It is stored as Fernet ciphertext in `oauth_clients` (its own HKDF context, so it can never be decrypted as a user token), is never returned to the browser (ID masked, secret never shown), can be replaced/removed without a restart (15-second cache so the API and the three MCP processes pick it up) and every change is audited. `GOOGLE_CLIENT_ID/SECRET` and `SLACK_CLIENT_ID/SECRET` in `.env` remain a documented fallback for automated deployments; the database value wins. Google offers no client-less flow and no dynamic client registration, so one registered client is unavoidable.
1 Initiate	User clicks **Connect** in Streamlit → API `POST /oauth/{service}/start` signs a 5-minute *connect ticket* (JWT bound to user + service) → browser opens `mcp-<service>/oauth/start?ticket=…`. The MCP server verifies it, creates a random **state** + PKCE verifier stored in Redis (10 min, single-use, bound to user_id+service) and 302s to the provider (Google: *offline access, prompt=consent, S256 PKCE*; Slack: bot scopes).
2 Authorise	User signs in with **their own** Google/Slack account and grants the scopes below.
3 Callback	Provider → `mcp-<service>/oauth/callback`. The state is *consumed* (`GETDEL`: replay fails), service must match, the code is exchanged (client secret + PKCE verifier), **granted scopes are verified** (Google's granular consent lets users untick permissions → token revoked and rejected), tokens are encrypted and stored under the state's user. Tokens never reach the browser. The MCP server then 302s to Streamlit with `?oauth=<service>&status=ok` plus a 2-minute single-use *resume ticket* that restores the login (the redirect starts a new Streamlit session).
4 Use & refresh	Each MCP call → `get_access_token(user, service)` under a row lock (`SELECT … FOR UPDATE`, so concurrent calls don't double-refresh): if the Google access token expires within 60 s, the refresh token silently obtains a new one and re-encrypts it. If refresh fails (revoked), the connection is flagged **expired** (Redis flag, audited), tools return `not_connected`, the reply says so and the sidebar card shows **token expired — Reconnect**.
5 Disconnect	**Disconnect** in the sidebar → provider revocation endpoint (Google `/revoke`, Slack `auth.revoke`) then the encrypted row is deleted; the card flips to **disconnected** immediately; audited.
Storage	**Fernet** (AES-128-CBC + HMAC-SHA256). The key is derived **per (user, service)** by HKDF-SHA256 from `TOKEN_ENCRYPTION_KEY`, so a blob copied into another user's row cannot be decrypted (unit-tested). Uploaded files use a separate HKDF context.

Service	Exact scopes requested	Justification (minimal for the tools)
Gmail	`openid email` · `gmail.send` · `gmail.readonly`	openid/email: show the connected address. send: confirmation/RM emails. readonly: list/read mail for 'summarise my emails' (no modify/delete).
Google Calendar	`openid email` · `calendar.events` · `calendar.freebusy`	events: create the invite. freebusy: availability without reading event details.

Service	Exact scopes requested	Justification (minimal for the tools)
Slack (bot)	`chat:write chat:write.public channels:read users:read users:read.email`	post alerts/DMs; post in channels the bot hasn't joined; resolve channel names; look up the RM by email.

Isolation: a token is only ever loaded with the verified subject of the call's JWT; state binds the flow to the initiating user; tests cover callback state validation, single use, wrong-service state, wrong-user storage, missing-permission rejection, redirect + resume ticket, and per-user encryption.

7. Streamlit Frontend Design

- **Structure:** sidebar = identity + **Integrations & OAuth** cards (connected / disconnected / token expired, account email or workspace, Connect/Reconnect/Disconnect) and preferences; tabs = Chat · Documents · Activity log · Approvals · Mock outbox (demo mode only) · Monitoring and Integrations setup (officers).
- **State:** only the session JWT and user dict live in `st.session_state` (per browser session, never shared); all data is fetched from the API on each rerun. A refresh would otherwise log the user out (Streamlit keeps state per websocket), so the JWT is also kept in a `SameSite=Strict` (Secure on https) cookie that is validated with `/auth/me` on load and deleted on sign-out; Streamlit cannot set HttpOnly cookies, an accepted trade-off noted in Section 11. OAuth redirects land on `/?oauth=…&status=…&resume=…`; `handle_oauth_return` exchanges the resume ticket (`POST /auth/resume`, single-use), shows success/error, and the card fragment refreshes.
- **Chat:** history with clear speaker attribution, live per-agent status ('analytics_agent — computed exactly over 931 rows'), token streaming, agent trace + MCP actions in an expander. **Aggregates render as metric cards + dataframe + bar chart** with a 'How this was computed' expander (periods, plan, repairs); a 'Sources' caption lists file / sheet / rows; 'Clear this conversation' deletes history.
- **Honest status:** a development switch (`MCP_MOCK`) can simulate the three services locally; the cards then read *simulated — no real account connected*, never a green 'connected'. It is off in the submission. **Files:** widget restricted to xlsx/csv, upload progress per file, list with upload time, status, row/chunk counts and error, delete. **Errors:** every API call goes through `ApiError` → friendly message (no stack traces); 409 when a conversation is still processing.

8. Tech Stack Justification

Technology and role	Why chosen over the alternatives	Limits in a banking context	Configured by / how to change
LLM: pluggable chat model (Ollama qwen2.5:14b local, any OpenAI-compatible API such as Groq, or Anthropic). Intake, planning, grounded answers.	No vendor lock-in; customer text can stay on-prem with Ollama. One LangChain chat-model interface with structured output. Over a single hard-wired API.	CPU inference takes minutes; small models need the plan-repair layer; hosted models send prompts off-site.	`LLM_PROVIDER, LLM_MODEL, OLLAMA_BASE_URL, OPENAI_BASE_URL, OPENAI_API_KEY, ANTHROPIC_API_KEY`; temperature 0 in `agents/helpers.py:llm()`
Embedding model: fastembed `bge-small-en-v1.5` (dense, 384-d) + `Qdrant/bm25` (sparse).	Local ONNX: no per-token cost and no data egress; hybrid keeps exact tokens ('750', 'BR001'). Over OpenAI embeddings (egress, cost) and dense-only.	Small model: lower recall on paraphrase; English only.	`EMBED_DENSE_MODEL, EMBED_SPARSE_MODEL, EMBED_DENSE_DIM`; baked into the image in `backend/Dockerfile`. Changing the model needs a re-index.
Vector DB: Qdrant 1.12. Stores both vectors per chunk, filters by owner.	Native dense + sparse hybrid with RRF and payload filters, so tenant scoping happens in the store. Over pgvector (no sparse / RRF) and FAISS (no filters, no persistence).	Payloads are plaintext at rest (use disk encryption); single node.	`QDRANT_URL, COLLECTION`; image tag in `docker-compose.yml`; vector params in `rag/store.py`
LangGraph (>= 0.4) + Postgres checkpointer. Agent orchestration and durable state.	Conditional edges, parallel `Send`, durable `interrupt()` for human-in-the-loop. Over a LangChain chain (no branching or persistence) and CrewAI / AutoGen (less control of state and failure paths).	Checkpoints grow; no pruning policy yet.	Pinned in `backend/requirements.txt`; topology in `agents/graph.py`; recursion limit in `agents/runtime.py`; uses `DATABASE_URL`
MCP library: official python SDK (FastMCP, streamable HTTP, stateless), `mcp >= 1.9`. Three tool servers.	Standard tool protocol; per-request headers carry the user JWT; each server scales separately. Over in-process SDK wrappers (no isolation) and stdio (no per-user headers).	One HTTP round-trip per tool call.	`MCP_GMAIL_URL, MCP_CALENDAR_URL, MCP_SLACK_URL, MCP_PORT, MCP_INTERNAL_SECRET`; tools in `mcp_servers/*.py`
API framework: FastAPI (async) with SQLAlchemy / asyncpg, redis.asyncio, httpx.	No blocking call in a handler (bcrypt, pandas and ONNX run in threads); OpenAPI docs for free. Over Flask / Django (sync-first).	No schema migrations (`create_all`).	`DATABASE_URL, REDIS_URL, JWT_SECRET`; uvicorn command in `docker-compose.yml`
Message broker / worker: arq on Redis. Ingestion, resume after approval, audit mirror.	Async-native and reuses the Redis already needed for locks and OAuth state. Over Celery (heavier, sync-oriented) and RQ (sync).	At-least-once delivery, so every job is idempotent.	`REDIS_URL`; `WorkerSettings` in `worker.py` (max_jobs 4, timeout 600 s, 3 tries)
Streamlit 1.65.0. The whole user interface.	Required by the brief; fast to build; fragments for live status polling.	Reruns the script on every interaction; cannot set HttpOnly cookies.	Pinned in `frontend/requirements.txt`; `API_URL`; server flags in `frontend/Dockerfile`
OAuth2 library: none; hand-written authorization-code + PKCE flow on httpx.	About 150 lines with full control: flow hosted on the MCP servers, scope verification, revocation, runtime-configurable client. Over Authlib / oauthlib, which hide the steps this document must explain.	Only Google and Slack are implemented.	Scopes in `oauth/providers.py:SERVICES`; client via Integrations setup or `GOOGLE_/SLACK_CLIENT_ID/SECRET`; redirect bases `MCP_PUBLIC_*`, `SLACK_REDIRECT_URI`

Technology and role	Why chosen over the alternatives	Limits in a banking context	Configured by / how to change
Token storage and encryption: Fernet + HKDF-SHA256 in Postgres (`cryptography`).	Authenticated encryption with a separate derived key per (user, service), and separate contexts for the app client secret and uploaded files; no extra service. Over a KMS / Vault (better key rotation in production).	Master key lives in the environment; rotating it forces every user to reconnect.	`TOKEN_ENCRYPTION_KEY`; derivation in `security.py`
Containerisation: Docker Compose, 9 services with health checks.	One command starts everything. Over Kubernetes (too heavy for this scope).	Single host, no high availability.	`docker-compose.yml` + `.env` (every variable listed in `.env.example`)
Third-party SDKs: pandas / openpyxl (tables), bcrypt, PyJWT.	pandas gives exact, fast aggregation over full files; openpyxl reads every sheet. Over DuckDB / SQL generation (a typed plan is safer than executing generated SQL).	Tables are loaded in memory (row cap 100k per sheet).	`MAX_UPLOAD_MB, MAX_TABLE_ROWS, FINANCIAL_YEAR_START_MONTH`

9. Multi-User Architecture

- Each conversation is its own LangGraph thread keyed `user_id:conversation_id`, built server-side from the authenticated JWT; every conversation/document/approval endpoint returns **404** for objects the caller doesn't own. State carries `user_id` from the session, so no node can act for another user.
- A Redis lock allows one in-flight run per thread; asyncio + arq give concurrency across users; `test_concurrent_users_do_not_leak` runs two users in parallel and asserts no cross-over in retrieval scope, MCP calls or answers. Vector queries are filtered by owner in Qdrant; aggregates and profile reads load only the caller's own (+ bank-wide) files.
- OAuth tokens: per-(user, service) rows, per-user derived keys, MCP verifies the JWT subject; contextvars attribute LLM/MCP audit rows to the right user.

10. Banking Domain Considerations

- **Sensitivity:** uploaded files and OAuth tokens encrypted at rest; local embedding/LLM option keeps data on-prem; secrets only in env; tokens never in browser, logs or audit payloads (MCP args logged by key only; LLM prompt/response truncated).
- **No hallucinated numbers:** grounded prompts with citations, exact pandas for aggregates and profiles, explicit 'not found'. **Regulatory:** large-transfer threshold read from the compliance CSV; high-risk requests need officer approval (four-eyes); the assistant never moves money.
- **Compliance logging:** append-only `audit_events` for files, agent steps, MCP calls, LLM calls, OAuth events, approvals, queryable by kind via `/activity` and summarised in the Monitoring tab; an audit summary is mirrored to Slack. **Prompt-injection:** email/Slack/file text is treated as data; tool choice is rule-based; Slack control sequences escaped.

11. Tradeoffs Made Deliberately & What I Would Do With More Time

Deliberate tradeoffs

- **Rule-based tool selection** instead of free LLM tool-calling: less flexible, but auditable and immune to injected instructions.
- **Exact aggregation over embeddings:** one more LLM call and a schema-bound plan language, but correct totals. Not built: free-form SQL generation (a validated plan DSL is safer than executing generated code).
- **Hand-rolled OAuth, Fernet+env master key, single-host Compose**: simple and explainable; no KMS, no HA, no schema migrations (`create_all`), the session cookie is readable by page scripts (no HttpOnly without a reverse proxy in front of Streamlit), no rate limiting beyond login throttling.
- **Data scope:** files are scoped per uploader (customer or bank-wide), not per `customer_id` row: an uploaded multi-customer transaction file is visible to its uploader in full.

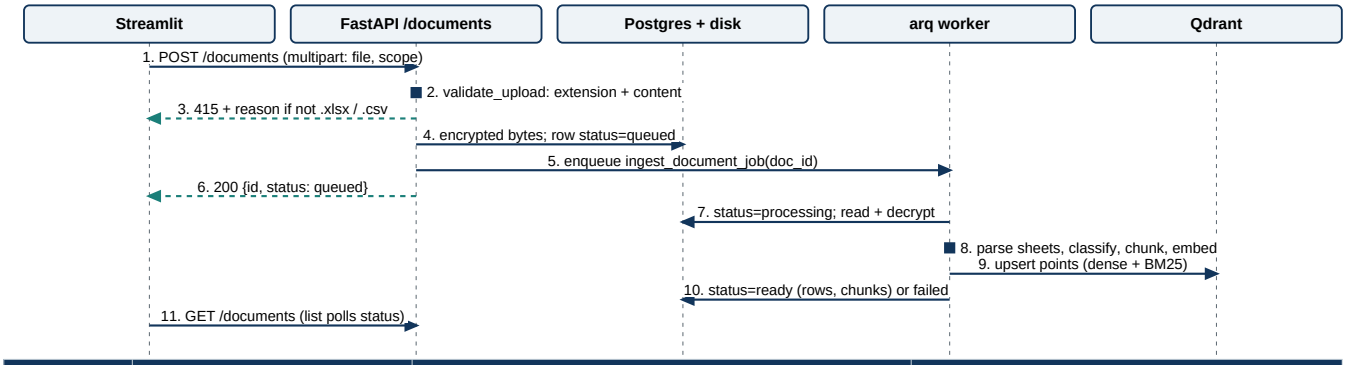
With more time

- Map each login to a `customer_id` and enforce row-level security inside the aggregation engine; a KMS-backed key hierarchy with rotation; Alembic migrations; Qdrant payload encryption.
- Evaluation harness with a labelled query set (accuracy of plans/aggregates per model), plan caching for repeated questions, streaming partial tables, a small LLM for planning and a larger one for prose, and OpenTelemetry traces into the monitoring page.
- Token-expiry proactive refresh job, multi-workspace Slack, CI pipeline with a live OAuth contract test against provider sandboxes.

Appendix A. Process Documentation

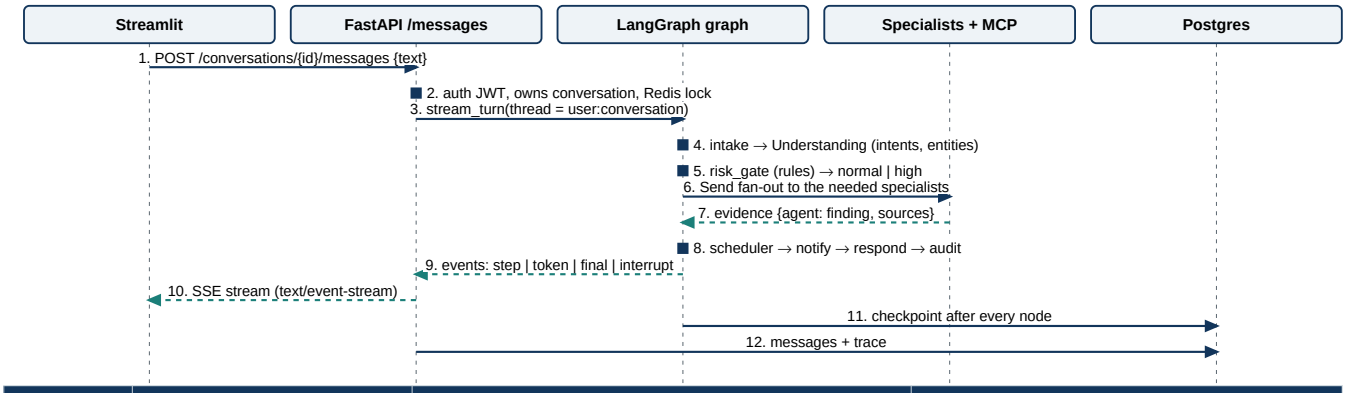
Each process below runs from its trigger to completion. Solid arrows are calls, dashed arrows are returns, a square is a step inside one component. Sections 2-6 explain *why* each process is designed this way; this appendix records *what happens*, the data exchanged and how each step fails.

A.1 File ingestion and validation



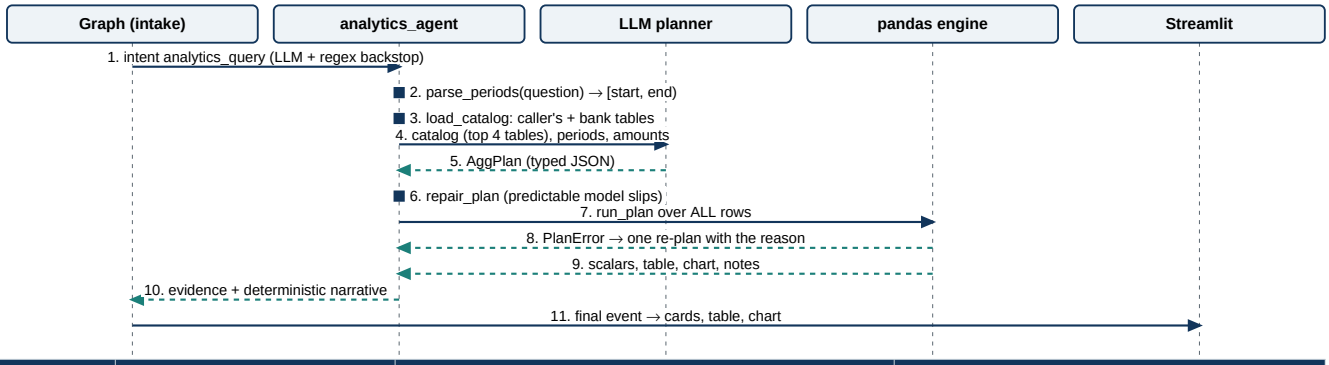
Step	Components	Data passed (format)	Error handling
1 Select	Streamlit `st.file_uploader (type=['xlsx','csv'])`	File bytes and name as multipart/form-data with `scope` (customer | bank); session JWT in the header.	Other extensions cannot be picked in the widget; API errors are shown as a message, never a stack trace.
2 Validate	FastAPI `upload` → `validate_upload`	(filename, bytes) → extension. xlsx must be a zip containing `xl/workbook.xml`; csv must not be binary or a zip.	Wrong type or renamed PDF → **415** with the reason and a `file_rejected` audit row; over MAX_UPLOAD_MB → 413; nothing is stored.
3 Store	FastAPI → disk, Postgres	Fernet ciphertext at `UPLOAD_DIR/<uuid>.<ext>`; `documents` row {owner_id, scope, filename, status=queued}; `file_uploaded` audit row.	Server-generated file name (no path traversal); a storage error returns a generic 500.
4 Queue	arq on Redis	Job `ingest_document_job(doc_id: str)`.	At-least-once, 3 tries; re-running is safe because old points are deleted first and point ids are deterministic.
5 Parse + chunk	Worker: `load_tables`, `classify_table`, `build_chunks`	list[(sheet, DataFrame of strings)] → list[Chunk {text, kind: schema | row | summary, sheet, row_start, row_end}].	Unreadable workbook or more than MAX_TABLE_ROWS → status *failed* with the error text, `file_failed` audit row.
6 Embed + index	Worker: fastembed → Qdrant	Batches of 64: dense float[384] and sparse {indices, values}; point payload {text, kind, table_kind, filename, sheet, scope, owner_id, row range}.	Any exception marks the document *failed*; the user can delete and re-upload.
7 Ready	Postgres, Streamlit file list	`documents.status=ready`, `n_rows`, `n_chunks`, `kinds`; `file_indexed` audit row.	A failed file shows its error in the list and is never searched.

A.2 User query to agent routing



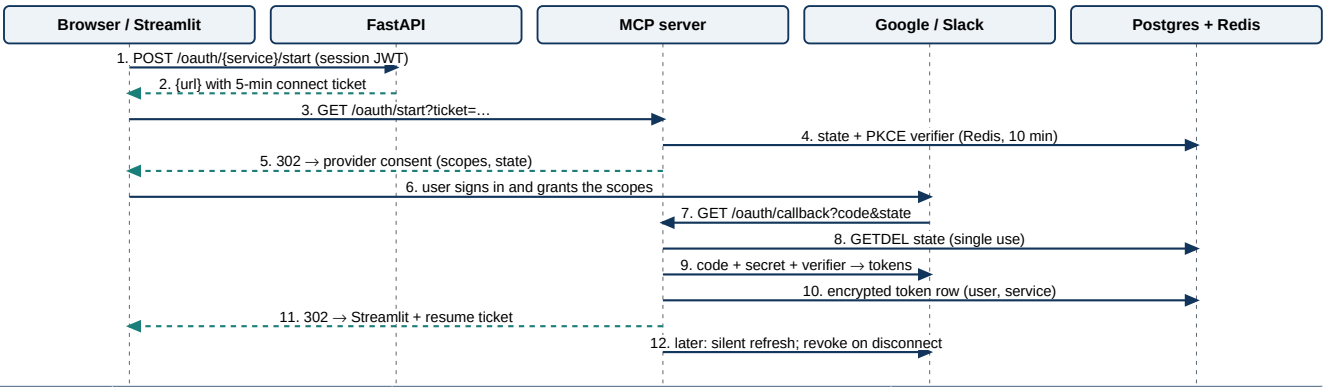
Step	Components	Data passed (format)	Error handling
1 Send	Streamlit chat input → `Api` client	JSON {text, max 4000 chars}; `Authorization: Bearer` session JWT.	`ApiError` → friendly message; 401 returns to sign-in.
2 Admit	FastAPI `send_message`	User id from the JWT only; lock `lock:<user>:<conversation>` in Redis.	404 if the conversation is not the caller's; 409 while a previous message is processing; lock released in `finally`.
3 Understand	`intake` (LLM structured output)	Last 6 messages → `Understanding` (pydantic): intents[], loan_type, tenure, amount, appointment date/time…; dates and aggregate wording corrected in code.	Ambiguous → `clarify` interrupt (max 2 rounds); LLM failure → generic error event.
4 Gate	`risk_gate` (rules; threshold read from the compliance file)	risk {level, reasons[], threshold}.	High risk → approval row, Slack alert, durable pause for an officer; out of scope → straight to the reply.
5 Route	`dispatch` conditional edge	List of `Send` objects; each specialist returns evidence[agent] = {summary, sources}, merged by a reducer.	A specialist without data returns an explicit 'no file / not found' finding, never a guess.
6 Act	`scheduler`, `notify` (A.5)	appointment {status, when, reference}; actions[{service, kind, status, detail}].	Failed or skipped actions are recorded and the turn continues.
7 Reply	`respond` → SSE → Streamlit	JSON events; final = {answer, trace, actions, analytics, sources}.	Exception → {type: error} with a generic message.
8 Persist	`audit` node, checkpointer	`messages` row with trace JSON; audit rows (A.6); checkpoint in Postgres.	Audit writes never raise; state survives a restart.

A.3 Aggregated query handling



Step	Components	Data passed (format)	Error handling
1 Detect	`intake` + `AGG_WORDS` backstop	`intents` contains `analytics_query`.	If the planner says 'not an aggregate', the agent answers by grounded retrieval instead.
2 Period	`periods.parse_periods` (code, not the model)	list[Period {start, end exclusive, label}], e.g. Q1 2026, last quarter, financial year.	No period → all dates, stated in the answer; model-written date filters are discarded.
3 Catalog	`load_catalog`, `describe_catalog`	{table name: DataFrame}; text description (columns, types, sample values, join hints) of the 4 most relevant tables.	No files → an explicit 'no data uploaded' finding.
4 Plan	LLM structured output	`AggPlan`: query {table, date_column, filters[], group_by[], metrics[], join, top_n…}, denominator, compare_previous_period.	The model never computes a number or writes a date.
5 Repair	`aggregate.repair_plan`	The same plan plus a list of human-readable repairs.	Every repair is shown under 'How this was computed'.
6 Execute	`aggregate.run_plan` (pandas, worker thread)	Filter → join → group → aggregate over the full files; `ExecResult` {rows_matched, coverage, scalars}.	Unknown table / column or bad value → `PlanError` → one re-plan; then 'I couldn't compute that exactly' (no estimate).
7 Present	`narrate`; Streamlit `render_analytics`	Payload {scalars, table, chart, detail, notes, sources, narrative} → metric cards, dataframe, bar chart.	Zero rows or a period outside the data's dates is stated; the LLM does not rewrite the numbers.

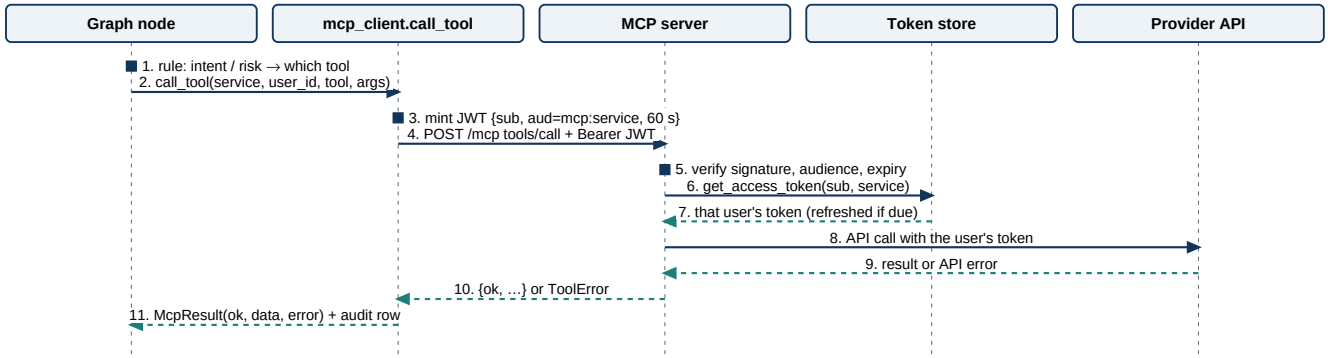
A.4 OAuth2 token lifecycle (Gmail, Calendar and Slack use the same flow)



Step	Components	Data passed (format)	Error handling
1 Initiate	Connect button → API → MCP `/oauth/start`	Connect ticket: JWT {sub=user, aud=mcp:<service>:connect, 5 min}. Redis: random state (32 bytes) → {user_id, service, PKCE verifier}.	No client configured → 'Not set up yet'; forged or expired ticket → 'Link expired'.
2 Authorise	Provider consent screen	Scopes. Gmail: openid email gmail.send gmail.readonly. Calendar: openid email calendar.events calendar.freebusy. Slack: chat:write chat:write.public channels:read users:read users:read.email.	User cancels → `error=access_denied` is shown in the app.
3 Callback	MCP `/oauth/callback` → `flow.finish`	Query: code, state. State is consumed with GETDEL and must match the service.	Missing, unknown, replayed or wrong-service state → rejected, nothing stored, `oauth_failed` audit row.
4 Exchange	`providers.exchange_code` (server-side httpx)	Form POST: code, client_id, client_secret, redirect_uri, code_verifier → access_token, refresh_token, expires_in, scope.	Provider error → message; a scope the user unticked → token revoked and rejected; no refresh token → rejected.
5 Store	`tokens.save_connection`	Fernet ciphertext of {access_token, refresh_token, expires_at}; key = HKDF-SHA256(TOKEN_ENCRYPTION_KEY, user, service); one row per (user, service) with scopes and account label.	A blob copied to another user's row cannot be decrypted (unit-tested).
6 Return	302 to Streamlit	`?oauth=<service>&status=ok&resume=…` (2-min single-use ticket) → `POST /auth/resume` restores the login.	Invalid ticket → the user signs in again. Tokens are never in a URL or the browser.
7 Refresh	`tokens.get_access_token` on every tool call	Row lock (`FOR UPDATE`); if the access token expires within 60 s, a refresh_token grant returns a new one, re-encrypted; `oauth_refreshed` audit row.	Refresh fails → *expired* flag, tools return `not_connected`, card shows 'token expired – Reconnect'.

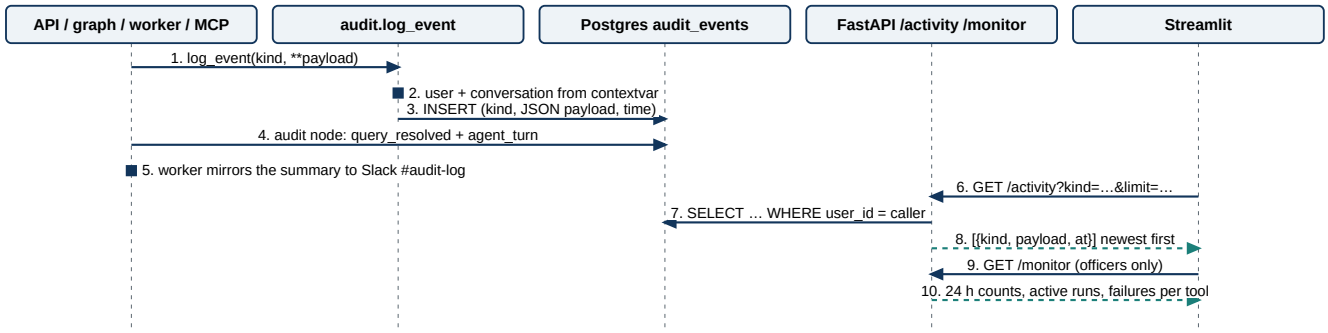
Step	Components	Data passed (format)	Error handling
8 Revoke	`DELETE /oauth/{service}`	Google `/revoke` or Slack `auth.revoke`, then the row is deleted; `oauth_disconnected` audit row.	Provider revocation is best-effort; the local delete always happens and the card flips at once.

A.5 MCP tool invocation



Step	Components	Data passed (format)	Error handling
1 Decide	`scheduler`, `notify`, `inbox_agent`, `request_approval`, audit job	Fixed mapping from intents, risk and booking result to a tool and its args, e.g. `send_email {to[], subject, body}`.	Service not connected → action 'skipped' with a 'connect it in Settings' message; no call is made.
2 Once	`helpers.once` (Redis)	Key `once:<turn>:<action>`.	A node re-run after a resume or retry does not send the email or post twice.
3 Identify	`make_internal_token`	HS256 JWT {sub=user_id, aud=mcp:<service>, exp 60 s, jti} in the Authorization header; args as JSON-RPC over streamable HTTP. Tools take no user or token argument.	—
4 Authenticate	MCP `common.authed`	Verified sub → that user's decrypted access token.	Bad or expired JWT → 'unauthorized'; no connection → `not_connected:<service>`.
5 Execute	Tool function → Gmail / Calendar / Slack API	Provider JSON → {ok: true, ids…}.	401/403 → not_connected; other → `google_api_error` / `slack_error:<code>`; missing Slack channel → general channel; server down or 30 s timeout → `mcp_unreachable`.
6 Report	`McpResult` → state.actions → reply and UI	{service, kind, status: ok | failed | skipped, detail}.	The failure is stated in the reply and the Activity log; the turn never aborts; `mcp_tool_call` audit row (tool, ok, error, ms, argument keys only).

A.6 Audit logging



Step	Components	Data passed (format)	Error handling
1 Emit	Every component → `audit.log_event`; `AuditCallback` for LLM calls	Kinds: file_uploaded / rejected / indexed / failed; agent_turn (steps[agent, status, detail, ts]); llm_call (model, prompt, response, ms; clipped); mcp_tool_call (service, tool, ok, error, ms, arg_keys); oauth_connected / refreshed / refresh_failed / disconnected / failed; oauth_client_saved / removed; approval_decided; query_resolved.	`log_event` never raises: an audit failure is logged and the user's request continues. Tokens and secrets are never written; MCP arguments are logged by key only.
2 Attribute	`audit_ctx` ContextVar, set per turn	user_id and conversation_id attached to LLM and MCP rows.	Without context the row is stored with no user.
3 Store	Postgres `audit_events`	id, user_id, conversation_id, kind (indexed), payload (JSON), created_at (indexed). Append-only: no update or delete route.	—
4 Mirror	`audit` node → arq `post_audit` → Slack MCP	Structured 'Customer query resolved' card to the user's audit channel.	Best-effort; the database row is already durable.
5 Query	`GET /activity?kind=a,b&limit=n&all=true`	JSON list, newest first. Customers see only their own events; officers may pass `all`.	401 without a session; `all` is ignored for customers.
6 Display	Streamlit Activity log and Monitoring tabs	Table of time, agent or event, action, status; filter by event type. Monitoring: events in 24 h, active runs, queue depth, LLM latency, failures per tool.	An API error shows a message, not a stack trace.